Projet : Banking API Modulaire

Document de spécifications techniques et fonctionnelles

A. Fiche d'identification

Champ	Valeur
Auteur	Zangue Sandji Roger Paolo Rayan
Matricule	23U2344
Établissement	L3 ICT4D
Unité d'enseignement	ICT304 — Software Testing and Quality Assurance
Livrable	API REST déployée + documentation Swagger + suite de tests
Dépôt source	https://github.com/PAOLO-SANDJI/-banking-api-paolo
URL de production	https://banking-api-paolo.onrender.com

B. Vision et enjeux

L'objectif de ce projet est de concevoir une **API bancaire éprouvée par le test** — pas seulement de la faire fonctionner, mais de la rendre **démonstrable** (Swagger interactif), **vérifiable** (suite automatisée) et **évolutive** (architecture modulaire découpée en couches).

Enjeux pédagogiques couverts

1. Rédiger des spécifications exploitables (template de cas de tests ICT304).
2. Appliquer le principe du **defensive testing** (conditions normales + anormales).
3. Tracer chaque exigence vers son ou ses cas de tests (**matrice RTM**).
4. Automatiser l'exécution des tests contre un environnement réel (Render).

C. Cartographie de l'architecture

```
                    ┌──────────────────────┐
                    │     Client (Swagger)     │
                    └──────────┬───────────┘
                               │ HTTP/JSON
                               ▼
┌──────────────────────────────────────────────────────────────┐
│                    index.js (entry point)                    │
│                    ▼                                         │
│               src/app.js (Express setup)                     │
│   ┌──────────────────┼──────────────────────────┐           │
│   ▼                  ▼                          ▼           │
│  routes/general.js   routes/comptes.js    routes/transactions │
│                      │                          │           │
│                      ▼                          ▼           │
│              services/compteService   services/transactionSvc │
│                      │                          │           │
│                      └──────────┬───────────────┘           │
│                                 ▼                            │
│                         services/store.js                    │
│                          (mémoire vive)                      │
└──────────────────────────────────────────────────────────────┘
```

Rationale des couches

- **routes/** : traduit HTTP ↔ domaine (validation syntaxique uniquement).
- **services/** : règles métier pures, testables sans serveur HTTP.
- **utils/** : helpers de formatage FCFA / dates / réponses normalisées.
- **store.js** : point unique de persistance — remplaçable par une vraie BDD sans toucher aux routes.

D. Portée fonctionnelle

D.1 — Périmètre inclus

```
✓ Création d'un compte bancaire (UUID côté serveur)
✓ Listing et consultation individuelle
✓ Dépôts et retraits (avec validations strictes)
✓ Historique filtré par compte
✓ Suppression définitive avec cascade sur les transactions
✓ Documentation Swagger auto-générée depuis les annotations JSDoc
```

D.2 — Hors périmètre (assumé)

```
✗ Authentification / autorisation
✗ Persistance durable (base de données)
✗ Gestion multi-devises
✗ Transferts entre comptes
✗ Intérêts, frais, agios
```

E. Contrat d'API

E.1 — Endpoints exposés

```
GET    /                              → Accueil + lien docs
GET    /api-docs                      → Swagger UI interactif

POST   /api/comptes                   → Créer un compte
GET    /api/comptes                   → Lister tous les comptes
GET    /api/comptes/:id               → Consulter un compte
DELETE /api/comptes/:id               → Supprimer (cascade)

POST   /api/comptes/:id/depot         → Déposer des fonds
POST   /api/comptes/:id/retrait       → Retirer des fonds
GET    /api/comptes/:id/transactions  → Lire l'historique
```

E.2 — Enveloppe de réponse normalisée

Toute réponse métier suit le contrat :

```
{
  "succes": true | false,
  "message": "Texte lisible par un humain",
  "donnees": <objet | tableau | null>
}
```

Les erreurs de validation d'entrée (400 sur POST /api/comptes) retournent un objet `{ erreur: "..." }` hérité du format legacy Express.

F. Règles métier formalisées

Code	Règle	Pénalité si violée
R01	Le solde initial d'un compte est de 0 FCFA	N/A
R02	`nom` et `prenom` sont requis et non vides	HTTP 400
R03	Un montant doit être `typeof === "number"`	HTTP 400
R04	Un montant doit être strictement supérieur à 0	HTTP 400
R05	Un retrait ne peut pas rendre le solde négatif	HTTP 400
R06	Un ID compte inconnu retourne HTTP 404	HTTP 404
R07	La suppression d'un compte supprime ses transactions	N/A (invariant)
R08	Les IDs (comptes et transactions) sont des UUID v4 serveur	N/A

G. Besoins (tableau compact)

Fonctionnels

```
BF01  Créer un compte                    ★★★ critique
BF02  Lister les comptes                 ★★★ critique
BF03  Consulter un compte par ID         ★★★ critique
BF04  Effectuer un dépôt                 ★★★ critique
BF05  Effectuer un retrait               ★★★ critique
BF06  Historique des transactions        ★★☆ important
BF07  Supprimer un compte (cascade)      ★★☆ important
```

Non-fonctionnels

```
BNF01 Réponse < 500ms sur plan Render Free (hors cold-start)
BNF02 Validation stricte des entrées (defensive)
BNF03 Codes HTTP conformes REST
BNF04 Retrait impossible si solde insuffisant
BNF05 OpenAPI 3.0 + Swagger UI accessibles
BNF06 Séparation des préoccupations (routes / services / utils)
BNF07 Cascade de suppression des données liées
```

H. Stack technique

```
Runtime          | Node.js ≥ 18 (API fetch native)
Framework HTTP   | Express 5
Docs             | swagger-jsdoc + swagger-ui-express (OpenAPI 3.0)
Persistance      | En mémoire (tableaux partagés via store.js)
Hébergement      | Render — Free Web Service
Tests            | Runner natif Node.js (aucune dépendance test)
PDF              | marked (Markdown → HTML) + Chrome headless
```

I. Stratégie de test

I.1 — Approche

Tests **boîte noire** contre l'API déployée, organisés par **module fonctionnel** plutôt qu'en un seul fichier monolithique. Chaque module (`tests/<domaine>.test.js`) exporte une fonction `run()` consommée par l'orchestrateur `tests/runner.js` .

Avantages pratiques :

- Exécution isolée d'un module (`node tests/comptes.test.js` ne marcherait qu'avec un runner local mais l'orchestration permet de cibler).
- Rapport JSON généré dans `reports/test-report.json` .
- Statuts automatiquement reportés dans le présent document (sections I.3 → I.8).

I.2 — Template (ICT304 Template 1)

Les colonnes utilisées : **Test Case ID / Description / Input / Attendu / Statut**.

Les numéros TC-1xx suivent un schéma par domaine :

- `001-002` → Général

- `101-111` → Comptes (CRUD)
- `201-210` → Dépôts
- `301-310` → Retraits
- `401-405` → Historique
- `501-505` → Suppression
- `601-603` → Intégration

I.3 — Général

ID	Description	Input	Attendu	Statut
TC-001	Endpoint racine répond JSON	GET /	200 + message + auteur	Pass
TC-002	Swagger UI chargée	GET /api-docs	200 + HTML Swagger	Pass

I.4 — Comptes (création, listing, consultation)

ID	Description	Input	Attendu	Statut
TC-101	Création valide	`{nom:"Sandji",prenom:"Paolo"}`	201 + UUID + 0 FCFA	Pass
TC-102	Nom manquant	`{prenom:"Paolo"}`	400	Pass
TC-103	Prénom manquant	`{nom:"Sandji"}`	400	Pass
TC-104	Corps vide	`{}`	400	Pass
TC-105	Nom vide	`{nom:"",prenom:"P"}`	400	Pass
TC-106	Prénom vide	`{nom:"S",prenom:""}`	400	Pass
TC-107	IDs uniques	2 créations	UUIDs différents	Pass
TC-108	Liste retourne un array	GET /api/comptes	200 + array	Pass
TC-109	Consultation ID valide	GET /api/comptes/:uuid	200 + données	Pass
TC-110	Consultation ID inexistant	UUID jamais créé	404	Pass
TC-111	Consultation ID malformé	GET /api/comptes/abc	404	Pass

I.5 — Dépôts

ID	Description	Input	Attendu	Statut
TC-201	Dépôt valide	`{montant:5000}`	200 + solde à jour	Pass
TC-202	Dépôt sur compte inexistant	ID inconnu	404	Pass

ID	Description	Input	Attendu	Statut
TC-203	Montant négatif	`{montant:-100}`	400	Pass
TC-204	Montant nul	`{montant:0}`	400	Pass
TC-205	Montant string	`{montant:"5000"}`	400	Pass
TC-206	Montant absent	`{}`	400	Pass
TC-207	Montant null	`{montant:null}`	400	Pass
TC-208	Dépôts successifs (cumul)	1000 puis 2000	Solde = 3000.00 FCFA	Pass
TC-209	Dépôt très grand	`{montant:1e9}`	200	Pass
TC-210	Dépôt décimal	`{montant:1234.56}`	200 + "1234.56 FCFA"	Pass

I.6 — Retraits

ID	Description	Input	Attendu	Statut
TC-301	Retrait valide	solde 10000 − 3000	solde 7000	Pass
TC-302	Retrait compte inexistant	ID inconnu	404	Pass
TC-303	Solde insuffisant	solde 1000 − 5000	400 "Solde insuffisant"	Pass
TC-304	Retrait = solde (limite)	solde 5000 − 5000	200 + solde 0	Pass
TC-305	Montant négatif	`{montant:-500}`	400	Pass
TC-306	Montant nul	`{montant:0}`	400	Pass
TC-307	Montant absent	`{}`	400	Pass
TC-308	Montant non numérique	`{montant:"abc"}`	400	Pass
TC-309	Retraits successifs	2 × 500 sur 1000	solde 0	Pass
TC-310	Retrait sur solde 0	`{montant:100}`	400	Pass

I.7 — Historique

ID	Description	Input	Attendu	Statut
TC-401	Historique vide	compte neuf	200 + []	Pass
TC-402	Après 1 dépôt	1 dépôt 5000	200 + 1 txn depot	Pass
TC-403	Historique mixte	1 dépôt + 1 retrait	200 + 2 txns	Pass

ID	Description	Input	Attendu	Statut
TC-404	Compte inexistant	ID inconnu	404	Pass
TC-405	Isolation entre comptes	A txns, B vide	B reste vide	Pass

I.8 — Suppression

ID	Description	Input	Attendu	Statut
TC-501	Suppression valide	ID UUID existant	200 + compteSupprime	Pass
TC-502	Suppression compte inexistant	UUID non créé	404	Pass
TC-503	GET après suppression	DELETE puis GET	404	Pass
TC-504	Cascade transactions	2 txns + DELETE	transactionsSupprimees = 2	Pass
TC-505	N'affecte pas les autres comptes	A + B, DELETE A	B toujours accessible	Pass

I.9 — Intégration

ID	Description	Input	Attendu	Statut
TC-601	Flux complet	Création + dépôt + retrait + hist.	Chaîne cohérente	Pass
TC-602	Créations concurrentes (×5)	`Promise.all`	5 UUIDs uniques	Pass
TC-603	Dépôts concurrents	2 × 1000 en parallèle	solde 2000	Pass

---

J. Matrice de traçabilité (RTM)

Conformément au cours ICT304, chaque exigence est tracée vers son ou ses tests.

```
┌──────┬──────────────────────────────────┬──────────────────────────┐
│ Req. │ Libellé                          │ Tests associés           │
├──────┼──────────────────────────────────┼──────────────────────────┤
│ BF01 │ Créer un compte                  │ TC-101 → TC-107          │
│ BF02 │ Lister les comptes               │ TC-108                   │
│ BF03 │ Consulter un compte              │ TC-109 → TC-111          │
│ BF04 │ Effectuer un dépôt               │ TC-201 → TC-210          │
│ BF05 │ Effectuer un retrait             │ TC-301 → TC-310          │
│ BF06 │ Historique transactions          │ TC-401 → TC-405          │
│ BF07 │ Supprimer un compte              │ TC-501 → TC-505          │
│ BNF02 │ Validation stricte des entrées  │ TC-102-106, 203-207,     │
│      │                                  │ 305-308                  │
│ BNF03 │ Codes HTTP REST                 │ Tous les TC              │
│ BNF04 │ Solde jamais négatif            │ TC-303, TC-310           │
│ BNF05 │ Documentation accessible        │ TC-001, TC-002           │
│ BNF07 │ Cascade de suppression          │ TC-504, TC-505           │
└──────┴──────────────────────────────────┴──────────────────────────┘
```

K. Glossaire

Terme	Définition contextuelle
Cascade	Suppression automatique des données dépendantes (transactions d'un compte supprimé).
Defensive testing	Méthode de test couvrant explicitement les entrées invalides/anormales.
Cold-start	Latence de premier appel après mise en veille Render (plan Free).
Enveloppe	Format uniforme `{succes, message, donnees}` appliqué à toutes les réponses.
RTM	Requirement Traceability Matrix — lie exigences et cas de tests.
UUID v4	Identifiant aléatoire de 128 bits, garanti unique en pratique.

L. Synthèse d'exécution

```
┌──────────────────────────────────────────────────────────┐
│  Total de cas de tests     : 46                          │
│  Exécutés                  : 46 / 46                     │
│  Pass                      : 46                          │
│  Fail                      : 0                           │
│  Couverture                : normale + anormale + limites│
│                              + intégration concurrente   │
│  Environnement de test     : banking-api-paolo.onrender  │
│  Automatisation            : runner Node.js natif        │
└──────────────────────────────────────────────────────────┘
```

Document maintenu à jour automatiquement par `tests/runner.js` à chaque exécution de la suite.